

Computerpracticum Informatica Open Dag

1 Introductie

Welkom bij het computerpracticum voor de bachelor Informatica in Utrecht!
In dit practicum ga je zelf code schrijven om de video-game ‘Snake’ af te maken. Aan de hand van sub-opdrachten voeg je steeds een functionaliteit toe, totdat alles werkt.

Je hoeft niet helemaal vanaf niets te beginnen. We geven je al de code waarin de basis van de game is uitgewerkt:

- **SnakeGame.cs** is het startpunt van het programma en stuurt alle onderdelen van de game aan. Hier worden ook het speelveld, de slang, en het systeem dat de appels spawn aangemaakt. Voor dit practicum hoef je hier niet veel in aan te passen en je hoeft het ook niet helemaal te begrijpen.
- **Snake.cs** bevat de code om de slang te tekenen, en wat variabelen die je nodig gaat hebben om de slang te laten bewegen, zoals de posities en beweegrichting.
- **Grid.cs** berekent de afmetingen van het speelveld en tekent dit ook naar het scherm. Voor de opdracht hoef je hier niets in aan te passen, maar je kunt er wel wat nuttige functies in vinden, zoals om te checken of de slang zich buiten het speelveld bevindt.
- **Apples.cs** houdt alle appels in het spel bij. Op het moment ontbreekt de code nog die de appels laat spawnen.

Videogames werken met een game-loop. Dit betekent dat het programma continue berekent wat er verandert in de gamewereld en dit tekent naar het scherm. In de code zie je dit ook terug, doordat de meeste classes een Update en een Draw functie hebben. Voor Snake komt daar ook nog bij kijken dat dingen niet continu veranderen, maar dat er een vast tijdsinterval tussen stappen zit. Voor dit practicum is dit interval al uitgewerkt.

Natuurlijk hebben we niet de ruimte om hier *alle* programmeerconcepten uit te leggen. Als je vragen hebt over hoe iets werkt of er niet uitkomt kun je altijd een van de studenten die rondlopen om hulp vragen. Daarnaast is Google je beste vriend: als je een programmeerconcept opzoekt met ‘C#’ erachter, komt je uit op de documentatie van Microsoft¹ die je vaak verder kan helpen. Voor de beginnende programmeurs is hier² een tutorial die de basale programmeerconcepten uitlegt.

Succes met de opdracht :)

¹<https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>

²<https://www.w3schools.com/programming/index.php>

2 De slang bewegen

File: Snake.cs, functie `void HandleMovement()`.

Als je het programma opstart zie je dat er nog niet zo veel gebeurt. De eerste stap is dus om de slang te laten bewegen. In het Snake.cs bestand zie je bovenaan een aantal variabelen staan. Voor het bewegen van de slang zijn `positions` en `direction` van belang.

`direction` is een `Point`³, wat bestaat uit een X en Y, die allebei integers zijn (gehele getallen). Dit staat dus voor de *richting* waarin de kop van de slang beweegt in het grid.

`positions` is een lijst van `Points`, waarbij elk van die `Points` voor een *locatie* in het grid van een van de blokjes in het lichaam van de slang staat.

Stel je hebt een lijst met 3 elementen (dus een slang met een hoofd, X en 2 lichaamsstukjes, Y en Z) dan ziet de lijst er zo uit: `positions == [X, Y, Z]`. We tellen hier vanaf index 0, dus als we punt X (het hoofd) uit de lijst willen pakken doen we `positions[0]`. Zo kunnen we ook `positions[1]` doen om het voorste non-hoofd lichaamsdeel te pakken, enzovoort.

De eerste `Point` in de lijst is de kop van de slang, de tweede het lichaamsstuk dat daaraan vast zit, enzovoort. Om de `Point` op positie `i` uit de lijst te krijgen, gebruik je de notatie: `positions[i]`. De lengte van de lijst kun je krijgen met `positions.Count`.

Taak: Zorg dat elk stukje van de slang correct vooruit beweegt.

Hint 1: Denk eerst na hoe elk stukje slang vooruit hoort te bewegen na één timestep (ze bewegen niet allemaal in de richting van `direction`).

Hint 2: Gebruik een for-loop⁴.

Hint 3: Werk vanaf de staart naar de kop van de slang toe.

Hint 4: De posities in een lijst gaan van 0 tot `Count - 1`

³<https://learn.microsoft.com/en-us/dotnet/api/system.drawing.point?view=net-10.0>

⁴https://www.w3schools.com/cs/cs_for_loop.php

3 De slang sturen

File: `Snake.cs`, functie `void HandleInput()`.

Nu de slang kan bewegen willen we haar natuurlijk wel kunnen bijsturen. Hiervoor hebben we keyboard input nodig. In de `Update(...)` functie in `Snake.cs` wordt al `HandleInput()` aangeroepen, maar `void HandleInput()` (lager in het bestand) is nu nog leeg.

We willen dat wanneer je op een specifieke toets drukt, de `direction` wordt aangepast. Je kunt checken of een toets wordt ingedrukt met `Keyboard.GetState().IsKeyDown(k)`, waarbij `k` de naam van de toets is. Voor spatiebalk zou je op de plek van `k` dus `Keys.Space` zetten.

Taak: Maak `void HandleInput()` functioneel. Zorg dus dat de `direction` variabele wordt aangepast na keyboard input.

Hint 1: Hier heb je if-statements⁵ voor nodig

Hint 2: Bovenaan in `Snake.cs` staan al `Keys` gedefinieerd voor alle richtingen.

Hint 3: De slang kan geen 180° draai maken. Je moet dus iets verzinnen om te zorgen dat keyboard-input die dit wel vraagt genegeerd wordt.

4 Collision logica

File: `Snake.cs`, functie `void HandleCollisionCheck()`.

Zoals je bij het testen van de vorige opdrachten misschien al gemerkt hebt, kan de slang nog wel buiten het grid en door zichzelf heen bewegen. We willen natuurlijk dat de speler dan af gaat. In `Snake.cs` staat de variabele `died`. Wanneer deze `true` is, zorgt de game-manager er al voor dat de game-over state start.

Taak: Zorg ervoor dat de speler afgaat als de slang buiten het grid of in zichzelf probeert te gaan

⁵https://www.w3schools.com/cs/cs_conditions.php

Hint 1: Neem een kijkje tussen de helper-functies in `Snake.cs` en `grid.cs`

Hint 2: Je kunt een functie uit `grid.cs` aanroepen in `Snake.cs` met `grid.FunctieNaam()`

5 Let there be apples

File: `Apples.cs`, functie `void SpawnApple()`.

Als het goed is heb je nu een slang die je kan aansturen terwijl ze over het grid beweegt en die afgaat wanneer ze een muur of zichzelf raakt. Alleen... dat is nog niet zo'n interessante game. In de standaard single player versie van Snake is het doel om appels te eten die om de zoveel tijd op het grid spawnen. We kijken voor nu eerst even naar het spawnen van appels.

Wederom is de time management in `void Update()` van `Apples.cs` al geregeld. Eenzelfde soort timer wordt gebruikt als voor de slang, zodat alles met dezelfde timestep verandert. Daarnaast bepaalt `spawnRate` na hoeveel timesteps een nieuwe appel spawn. Het enige wat nog moet gebeuren, is de code in `void SpawnApple()` die bepaalt waar de appel verschijnt.

De appels worden bijgehouden in een lijst⁶ van Points, `apples`. Het enige wat je hoeft te doen is de positie in het grid toevoegen aan die lijst. Als je een willekeurig getal van 0 tot 5 wil hebben, kun je `SnakeGame.random.Next(6)` gebruiken.

Taak: Zorg ervoor dat `void SpawnApple()` een appel toevoegt aan `apples`

Hint 1: Appels kunnen niet zomaar op elke plek verschijnen, wat als een plek al bezet is door een andere appel of door de slang? `Snake.cs` bevat een functie die je hierbij kan helpen voor de slang. Misschien dat de code in die functie je ook kan inspireren voor de appels...

Hint 2: Je kunt een nieuwe positie aan de lijst met appels toevoegen met de functie `apples.Add()`

⁶<https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1?view=net-9.0>

6 Wanneer een appel gegeten word...

File: Snake.cs, functie `void HandleAppleEating()`.

Nu de appels gespawnt worden in het grid, moet je ze ook kunnen opeten met de slang. Je moet dus na elke beweging checken of je daarmee een appel eet en in dit geval moet de slang groter worden.

Taak: Zorg ervoor dat de slang appels kan opeten

Hint 1: Apples.cs heeft al een functie waarmee je de appel kan opeten.

Hint 2: Voordat we de slang bewegen, slaan we de positie van zijn staart al op. Misschien dat je die kan gebruiken voor het laten groeien van de slang.

7 Bonus

Goed bezig! Je game werkt nu net zoals het originele spel dat uitkwam in 1982. Mocht je dit in rap tempo voor elkaar hebben gekregen is het natuurlijk leuk om de game wat op te leuken en ingewikkelder te maken. Hieronder zijn een aantal ideeën om net dat stapje extra te zetten met je programmeer skills.

- De slang start nu elke keer in het midden van het veld, maar dit zorgt ervoor dat je een hele strategie vanaf het begin kan uitdenken. Probeer maar eens uit te vogelen hoe de slang bij elk nieuw spel op een random punt in het speelveld kan beginnen.
- Als de slang op een random locatie begint, kan het zijn dat zij meteen tegen een muur botst. Hoe kun je ervoor zorgen dat de slang het speelveld in beweegt bij elke willekeurige startlocatie?
- Er zit nu steeds evenveel tijd tussen de bewegingen van de slang. Het spel kan spannender worden als deze tijd steeds korter wordt, waardoor de slang steeds sneller gaat bewegen.
- De appels spawnen nu steeds na dezelfde hoeveelheid tijd. Het spel kan leuker worden als de appels steeds sneller spawnen.

- Maak de game multiplayer, zodat jij en een maatje een eigen slang hebben om te besturen. Welke toetsen gebruik je om de slang te besturen? Wat gebeurt er als de slangen elkaar raken?